

Activity 11: REST JSON and configuration automation

TGS-2023037854 | v7.0 | 13 September 2026 | 80 minutes

Goal

Validate REST status and JSON types, produce an idempotent plan and select a bounded agentic AI prompt.

Before you start

Python 3.9+, a plain text editor and this complete folder. Offline mode requires no extra packages, Internet or credentials. The compulsory core Packet Tracer construction is specified below and in topology.md; advanced extensions may need CML/IOS XE.

Evidence boundary

All JSON/CSV observations are synthetic training evidence. Passing the Python checker validates the fixture contracts only. No hardware, simulator, authentication server or API is configured or contacted.

Files

README.md, instructions.pdf, checklist.pdf, topology.md, case.json, faulty.json, reference.json, observations.csv, specimen.cfg, S1-baseline.cfg, R1-baseline.cfg

Working method

case.json is the working evidence; faulty.json resets the exercise and reference.json shows corrected synthetic evidence for self-study. Preserve your own sandbox observations separately.

Detailed step-by-step instructions

1. Construct and save the mandatory simulator topology

Start Cisco Packet Tracer 8.x or a current compatible version. File > New; use the bottom device palette, drag these models into the workspace and rename them via Config > Display Name: 1 x Router2911 (R1), 1 x Switch2960 (S1), 1 x PC-PT (PC1). Choose Connections > Copper Straight-Through for the table unless otherwise stated. Switch-to-switch links use Copper Cross-Over or Automatically Choose Connection Type. Click the first device and select its named interface; click the peer and select its named interface. Wait for links to initialize. Save as activity-start.pkt.

Device/port A	Device/port B	Cable
PC1 FastEthernet0	S1 FastEthernet0/1	Straight-through
S1 FastEthernet0/24	R1 GigabitEthernet0/0	Straight-through

Save both activity-start.pkt and a working activity-working.pkt; use File > Save As.

2. Assign endpoint addresses and inspect interface identifiers

Set Desktop > IP Configuration > Static on each endpoint.

Endpoint	Address	Mask	Gateway	DNS
PC1	10.10.10.10	255.255.255.0	10.10.10.1	none

For routers/switches open CLI. Answer no to the initial configuration dialog and press Enter. Use the exact interface names below; a 2911 uses GigabitEthernet0/0 through 0/2, a 2960/3560 uses FastEthernet0/1 through 0/24 and GigabitEthernet0/1 through 0/2.

```
enable
show ip interface brief
```

3. Install all peer baseline configurations

Paste the following device-specific blocks into each named device CLI one device at a time. Wait for the router/switch prompt between blocks. These are complete activity configurations for the stated construction. Do not apply configuration from a different activity. When copy asks for Destination filename, press Enter. Save the Packet Tracer working file after all blocks.

Device S1

```
enable
configure terminal
hostname S1
no ip domain-lookup
interface range FastEthernet0/1, FastEthernet0/24
    switchport mode access
    no shutdown
exit
end
copy running-config startup-config
```

Device R1

```
enable
configure terminal
hostname R1
no ip domain-lookup
interface GigabitEthernet0/0
    ip address 10.10.10.1 255.255.255.0
    no shutdown
exit
```

```
interface GigabitEthernet0/0
  description Users
exit
end
copy running-config startup-config

show running-config
show ip interface brief
copy running-config startup-config
```

4. Prove the healthy baseline with actual packets

Run every test below before introducing the fault. PC/Desktop > Command Prompt provides ping, ipconfig and arp. Use Simulation mode and Edit Filters to keep ARP, ICMP and the protocol under study; click Add Simple PDU for a ping and Capture/Forward to inspect hops. The first ping may lose one packet during ARP; repeat and record the second result. Do not mark a test passed until observed.

PC1 ping 10.10.10.1 succeeds throughout. R1 show interfaces GigabitEthernet0/0 shows Description: Users after repair. Compare saved configs: only the interface description changes. Then run the independent JSON checker, deliberately change enabled from boolean true to string "true" and observe schema rejection; restore boolean. Record an empty difference plan when actual fixture equals desired state. PT2911 does not expose a genuine RESTCONF API; CLI observations alone do not prove API operation.

After all healthy tests pass, File > Save As > activity-start.pkt to preserve the CONFIGURED healthy baseline, then File > Save As > activity-working.pkt before introducing the fault. The initial empty construction is not the rollback baseline.

Save baseline screenshots / CLI text with device names; File > Save.

5. Introduce and diagnose the controlled fault

Save the baseline first. Apply only the following fault to the specified device. Repeat the same traffic tests, capture the first failing hop/control and compare show output with the healthy baseline.

R1 configure terminal; interface GigabitEthernet0/0; description Wrong; end. Capture show interfaces GigabitEthernet0/0 showing the unexpected description. Export a sanitized running config by copying visible CLI text into a local file; do not include credentials.

Retain activity-start.pkt and baseline running configurations; work in activity-working.pkt.

6. Apply the narrow repair and re-test the network

Write a proposed description-only change and rollback before applying it. R1 configure terminal; interface GigabitEthernet0/0; description Users; end. Apply the same target a second time; the desired state stays Users and traffic remains healthy.

Repeat the baseline test matrix. Check surrounding controls as well as the repaired flow. how desired-state comparison, JSON types and bounded proposals prevent accidental broad configuration changes

Save actual show output and allowed/denied traffic results; copy running-config startup-config only after successful verification.

7. Rollback and rehearse recovery

Save the repaired file as activity-verified.pkt. To undo the exercise, open the saved healthy baseline activity-start.pkt after you have saved configurations into it at Step 3, or paste the saved pre-fault running configuration into a clean topology. For the specific repair, retain the exact old command/value rather than guessing a trunk list or ACL. Repeat one positive and one relevant negative test after recovery.

File > Save As > activity-verified.pkt; close working topology; reopen saved healthy

baseline.

8. Submit observed evidence and explain the forwarding mechanism

Submit the verified .pkt file, a cabling/address table, pre-fault and post-repair show outputs, screenshots of traffic tests, diagnosis and rollback notes. Explain how desired-state comparison, JSON types and bounded proposals prevent accidental broad configuration changes Keep synthetic checker output separate from real simulator observations.

Record simulator version, device models and tests actually completed.

9. Complete the separately labelled extension

For a genuine RESTCONF exercise, use trainer-owned CML/IOS XE with HTTPS/RESTCONF enabled by the trainer and a least-privilege read account. In an API client issue GET `https://<sandbox>/restconf/data/ietf-interfaces:interfaces` with Accept `application/yang-data+json`; keep TLS certificate validation enabled, authenticate at runtime and capture 200/body. A401 requires correcting authorized authentication, not fabricating a 200 fixture. Do not send PUT/PATCH/DELETE. Use the agent prompt in the reference fixture with sanitized evidence and no execution tools. Ansible check mode/Terraform plan remain proposals; inspect the diff and require human review before any separately authorized write.

Optional actual read-only Ansible execution on a trainer-owned SSH sandbox (not Packet Tracer): install ansible-core and cisco.ios collection in a trainer-managed environment; create a local inventory with host `sandbox ansible_host=<authorized-address> ansible_connection=ansible.netcommon.network_cli ansible_network_os=cisco.ios.ios`. Store credentials in Ansible Vault or pass `--ask-pass`; do not commit them.

Create `read-only.yml` with:

```
- hosts: sandbox
  gather_facts: false
  tasks:
    - name: Collect interface evidence
      cisco.ios.ios_command:
        commands:
          - show ip interface brief
          - show interfaces GigabitEthernet0/0
        register: observation
    - ansible.builtin.debug:
        var: observation.stdout_lines
```

Run `ansible-playbook -i inventory.ini read-only.yml --ask-pass`. Capture returned show output and device/version. This invokes only show commands; no `ios_config` task is supplied. The offline fixture uses a simplified `application/json` schema, while an actual RESTCONF response follows the namespaced YANG schema; do not substitute the simplified fixture for real API evidence.

Record supported platform, output observed or evidence-only limitation.

Untimed appendix: independent fixture analysis

These synthetic cases supplement the allocated hands-on time and do not configure the simulator.

Appendix 1. Establish the baseline

Open `activity11-rest-json-configuration-automation`; inspect `topology.md`, `case.json` and `observations.csv`. Identify the controls for: Validate REST status and JSON types, produce an idempotent plan and select a bounded agentic AI prompt.

```
python3 --version
```

Appendix 2. Run the faulty evidence case

Run the checker against case.json; exit status 1 is expected at this stage. Record each FAIL label and the calculated detail.

```
python3 checker.py case.json
```

Appendix 3. Propose the repair

Restore status 200, enabled as JSON true (without quotes), description Users and tls_verify true. Replace the unsafe prompt with the bounded prompt in reference.json and set allow_writes false, sanitized true. Never add real tokens to fixtures.

Open case.json in a plain text editor; preserve JSON types.

Appendix 4. Validate the corrected case

Save a separate working copy first. Run the checker after each narrow edit until all controls PASS; capture output as evidence. This validates fixtures only.

```
python3 checker.py case.json > evidence.txt
```

Appendix 5. Verify checker behavior

Run self-test; it must accept corrected reference evidence and reject the faulty fixture and the independent corrupted-control test.

```
python3 checker.py --self-test
```

Expected results and evidence

Corrected reference: all checks PASS, exit 0; faulty fixture: one or more FAIL, exit 1; self-test: PASS, exit 0.

Domain-specific computed fields appear in JSON detail.

Capture evidence.txt, corrected case.json and your diagnosis. For sandbox work record device/model, timestamps, actual show outputs, allowed/denied tests and rollback. Distinguish fixture results from real observations.

Troubleshooting and reset

Exit 1: inspect the named failed contract. Exit 2: restore valid JSON keys/types. Unsupported simulator commands must be documented; never invent results. Reset by copying faulty.json over case.json. Restore sandbox settings from the actual saved baseline.

Calculated corrected-fixture results

```
change_plan: {}  
second_run_plan: {}  
request_method: "GET"
```

Required PASS controls

- REST success and JSON media
- JSON interface schema
- desired state idempotent
- read request secure
- bounded agentic proposal

Transition practice

Refresh extension: agentic AI may propose actions, but this activity grants no write tools. The checker tests a defined exercise prompt contract and cannot prove that arbitrary model behavior is safe.

Configuration files

The complete per-device configurations are shown in the construction steps and included as named *-baseline.cfg files. specimen.cfg combines those same blocks with explicit device boundaries. Apply each block only to its named device. WRT300N uses GUI settings instead of IOS CLI.

References

Cisco CCNA v1.1 exam topics; Cisco IOS XE configuration guides; Python ipaddress documentation. Exact URLs and source links are in README.md.